

AI/ML + GenAI Interview Questions with Answers

For 2 Years Experience Candidates | July 2026

1. Machine Learning Fundamentals

Q: Explain bias-variance tradeoff with an example from your project.

Bias = error from wrong assumptions, like underfitting. Variance = error from sensitivity to training data, like overfitting. Tradeoff: Simple model = high bias, low variance. Complex model = low bias, high variance. Ex: In a loan default model, linear regression had high bias and missed patterns. Random Forest overfit small data. Tuned depth + cross-val to balance. Goal: Find sweet spot with validation curves.

Q: How do you handle imbalanced datasets? Compare SMOTE vs class weights vs focal loss.

1. Class weights: Penalize minority errors more in loss. Fast, no data change. Use in sklearn with `class_weight='balanced'`. 2. SMOTE: Synthetically oversamples minority by interpolating neighbors. Risk: creates noise if clusters overlap. 3. Focal loss: Down-weights easy examples, focuses on hard misclassified ones. Best for object detection. Rule: Try class weights first, then SMOTE if more data helps, focal loss for deep nets.

Q: What is data leakage? Give 3 real examples.

Leakage = when training data has info from future/test that won't exist in prod. Ex1: Scaling before train/test split — test stats leak into train. Ex2: Time series: using future stock price as feature to predict today. Ex3: Target encoding on full data before split. Prevention: Always split first, use pipelines, respect time order.

2. Deep Learning & NLP

Q: Explain how Transformers work. Why is self-attention better than RNN?

Transformers use self-attention: each token attends to all others in parallel via Query, Key, Value matrices. Output = weighted sum of Values. vs RNN: RNNs process sequentially, can't parallelize, struggle with long range due to vanishing gradients. Attention is $O(1)$ path length vs $O(n)$ for RNN. Hence faster training + better long context.

Q: What is LoRA/QLoRA for fine-tuning?

LoRA: Low-Rank Adaptation. Freezes base model, injects trainable rank-decomposition matrices into attention layers. Only 0.1% params update. QLoRA: LoRA + 4-bit quantization of base model. Lets you finetune 65B models on 1 GPU. Tradeoff: Slight accuracy drop vs full finetune, but 1000x cheaper.

3. Generative AI & LLMs — 2026 Focus

Q: What is RAG? Explain end-to-end pipeline with vector DB, chunking, reranking.

RAG = Retrieval Augmented Generation. Solves hallucination + knowledge cutoff. Pipeline: 1. Chunk docs: 256-512 tokens with overlap. 2. Embed chunks with model like BGE-large. 3. Store in vector DB: Pinecone/Qdrant. 4. User query -> embed -> top-k search. 5. Rerank with cross-encoder. 6. Inject top chunks into LLM prompt. 7. LLM answers with citations. Key: chunk size, embedding quality, reranker boost accuracy 20%+.

Q: How do you reduce hallucinations in LLMs? Give 4 techniques.

1. RAG: Ground answers in retrieved docs. 2. Constrain decoding: use JSON mode, function calling. 3. Self-consistency: Sample multiple answers, pick majority. 4. LLM-as-judge: Second LLM verifies facts. 5. Prompting: 'Answer only if in context, else say I don't know'. In prod, combine RAG + citations + guardrails.

Q: Explain quantization. GPTQ vs AWQ vs GGUF.

Quantization = reducing model weights from FP16 to INT4/INT8 to cut memory/latency. GPTQ: One-shot, layer-wise quantization, good for GPU. AWQ: Activation-aware, protects salient weights, better accuracy. GGUF: llama.cpp format, CPU optimized, runs 7B on laptop. Tradeoff: INT4 = 4x smaller, ~1-3% accuracy drop. Use Q4_K_M for best quality/size.

Q: What is function calling/tool use in LLMs?

LLMs can't do math/browse/call APIs natively. Function calling = LLM outputs structured JSON with function name + args when needed. Your code executes it, feeds result back to LLM. Ex: User: 'Weather in Bhubaneswar?' -> LLM: calls `get_weather(city='Bhubaneswar')` -> You run API -> LLM: 'It's 31°C and cloudy'. Used in agents, RAG, data analysis. OpenAI/Anthropic/Gemini all support it.

4. MLOps & System Design

Q: How would you design a system to serve a 7B LLM for 10k users/day?

Assume 500 tokens avg, 10k req/day = ~5M tokens. Need ~2 req/s peak. Stack: 1. Model: Llama-3.1-8B Q4_K_M on 1xA10 GPU with vLLM for continuous batching -> 40 tok/s. 2. Cache: Redis for frequent prompts. 3. Queue: SQS for async. 4. Streaming: SSE to reduce perceived latency. 5. Autoscale: 1-3 GPU nodes. Cost: ~\$300/month. Monitor: latency p95, token throughput, hallucination rate with eval set.

Q: What is model drift? How do you monitor and retrain?

Drift = prod data distribution changes vs training. Data drift: input features shift. Concept drift: $P(y|x)$ changes. Monitor: KL divergence, PSI for features. Track accuracy on live labeled sample. Alert if >10% drop. Retrain: Schedule monthly or trigger-based. Use MLflow for versioning. Shadow deploy new model before swap.

5. Hands-on Coding & SQL

Q: SQL: Find 2nd highest salary without LIMIT/OFFSET.

`SELECT MAX(salary) FROM employees WHERE salary < (SELECT MAX(salary) FROM employees);`
Alternative: Use `DENSE_RANK() OVER (ORDER BY salary DESC) = 2` in subquery. LIMIT/OFFSET fails if duplicates exist.

Q: Code: Implement TF-IDF for a corpus.

TF = term count / doc length. IDF = $\log(N / \text{docs_with_term})$. TF-IDF = TF * IDF. In sklearn: TfidfVectorizer. From scratch: Count vectorize each doc, compute IDF across corpus, multiply. Use: keyword extraction, cosine similarity for search.

6. Projects & Behavioral

Q: Walk me through an end-to-end ML project you owned. What failed?

STAR format: Situation: Churn prediction for SaaS. Task: Reduce 15% monthly churn. Action: Built XGBoost on 50 features, SHAP for explainability, deployed via FastAPI + batch scoring. Result: Cut churn to 11%, \$200k ARR saved. Failure: First model leaked data via future subscription_status. Caught in backtest. Learning: Always do temporal splits.

Q: If ChatGPT gives wrong answer for your domain, how would you fix it for users?

1. Add RAG with company docs to ground it. 2. Fine-tune on domain Q&A; pairs. 3. Add guardrails: 'I don't know' when confidence low. 4. Human-in-loop: flag uncertain answers for review. 5. Feedback button to collect corrections. Metric: Reduce hallucination rate from 12% to <3% on eval set.

7. 2026 Hot Topics

Q: SLMs: Phi-3, Gemma 2, Llama-3.1-8B. When to use vs 70B?

Small Language Models = 1B-8B params. 10x cheaper, 5x faster, run on phone/CPU. Use when: classification, extraction, simple chat, RAG where context has answer. Use 70B when: complex reasoning, code gen, creative writing. Strategy: Router model sends easy queries to Phi-3, hard to Llama-70B. Saves 80% cost.

Q: Vector DBs: Pinecone vs Weaviate vs pgvector vs Qdrant — tradeoffs.

Pinecone: Managed, fast, expensive at scale. Weaviate: Hybrid search, GraphQL, open-source. pgvector: Postgres extension, SQL joins with vectors, best if you already use Postgres. Qdrant: Rust, best filters + payload, good for on-prem. Pick: Startup MVP = Pinecone. Need filters + self-host = Qdrant. Already on Postgres = pgvector.

Tips for 2 YOE Candidates

- Always use metrics: Before/after accuracy, latency, cost, revenue impact.
- For GenAI, build 1 RAG demo + 1 agent demo. Put on GitHub.
- Know your tradeoff decisions: Why XGBoost not deep learning? Why 8B not 70B?
- System design: Think tokens/sec, cost per 1k requests, p95 latency.
- Stay current: Read Llama-3.1, DPO, FlashAttention-2 papers. Summarize in 1 min.